

Dynamic Poisson Factorization Replication

William Yang
yang7313@umn.edu

Jinming Chen
chen6386@umn.edu

ABSTRACT

Traditional recommender systems often use static models that may not fully capture changes in user preferences over time. Dynamic Poisson Factorization (dPF) was introduced to model these temporal dynamics, but its effectiveness on different varieties of datasets has not been thoroughly tested. In this work, we attempt to replicate and evaluate dPF on multiple datasets, including Netflix, Last.fm, and MovieLens, while comparing it to simpler baselines such as Poisson Factorization (PF) and Bayesian Personalized Ranking (BPR). Our results show that while dPF can improve recommendation ranking in some cases, its performance varies greatly depending on the dataset and evaluation metrics. We also discuss the challenges and limitations of replicating previous work when applying dynamic models to different recommendation scenarios.

1 INTRODUCTION

Historically, static models of recommender systems have limitations in sensing temporal dynamics. Static models, such as matrix factorization, assume fixed user preferences and item attributes over time.[5] Algorithms that use matrix factorization on implicit feedback for item recommendation are not well suited for ranking[9]. Other algorithms, such as Poisson Factorizations(PF) and Bayesian Personalized Ranking(BPR), are more suited for ranking-based tasks.[3] The drawback of a static model is its limitation in capturing the changing focus of users. The work by Charlin et al.(2015) addresses this gap by proposing a Dynamic Poisson factorization (dPF) framework, which integrates Kalman filtering to model evolving user-item interactions[2]. The model tracks user-item interaction temporal shifts as time slices. Although dPF performs better in specific tasks, its reproducibility across diverse datasets remains unexplored. In this study, we reuse and modify the core dPF’s algorithmic components and evaluate its performance on datasets, including Netflix, Last.fm, and MovieLens. By decomposing, deriving, and identifying ambiguous experimental steps from the original work and systematically testing parameters, our replication aims to verify dPF’s consistency, thereby providing actionable outcomes for adapting dynamic factorization models to other practical scenarios.

2 DATA PREPARATION

The original studies used the Netflix Prize and arXiv datasets, but the latter isn’t available publicly. Instead, we substituted arXiv with two additional datasets, MovieLens 100K and Last.fm 1k, for a total of three datasets. The Netflix Prize data set was later split into two datasets: the Netflix full, which contains 30% of the Netflix Prize dataset, and the Netflix time. This resulted in four different datasets, which were further split into a 70:30 ratio of train and test sets for the models. All data is binarized before being passed into the models. Additionally, the dataset being passed into the PF will have to be renamed into the specific names of UserId, ItemId, and Count.

2.1 Netflix Prize

The Netflix Prize dataset¹ was initially used in an open competition to find the best algorithm for predicting user ratings. It has 480K users and 17K items across over 100M ratings from 2000 to 2005. This high-quality benchmark dataset includes a sparsity challenge, suitable temporal dynamics, and implicit user feedback, making it well-suited for dPF.

2.1.1 Netflix Time. We attempted to follow the study process in creating Netflix Time; however, due to the lack of clarity, we instead followed the paper references Li, Bin et al outline on how they prepared the dataset.[8] The procedure outlines a process for filtering for active users across a time period. However, direct replication of their approach failed to reproduce similar data shapes. We adjusted the sequence of the operation to yield similar results as follows:

- (1) Select movies that were imported before 2002. We defined it as if the movie had at least one rating before 2002.
- (2) Select users registered before January 1st, 2002. By registered, we defined it as if the user left at least one rating before 2002. Based on that, we chose users with at least one rating after January 1st, 2005.
- (3) Filtered ratings only after 2002, the time period has the highest density of ratings.
- (4) Assign 16 equal time slices with the filtered ratings based on their timestamps, from January 1st, 2002, to December 31st, 2005. Each slice lasts 3 months.
- (5) If the users have at least 15 ratings during such a slice, we mark it as valid. Then, we calculate valid slices for the remaining users.
- (6) Further, we chose users with over 100 total ratings and at least four valid slices.
- (7) Finally, we end up with 6582 users and 4105 movies.
- (8) We chose the final rating from the last 3 time slices (9 months) as the test set; the rest are the train set.

2.1.2 Netflix Full. Considering our limitations on computing resources, we didn’t use all 100M ratings. Instead, we processed the data into a shape that contains a reasonable number of users and items while keeping the same sparsity of the original data. Our steps are as follows:

- (1) We picked a length of 30% of the original 480,189 users. That is 144,056 users.
- (2) We picked a length of 30% of the original 17,770 movies. That is 5331 movies.
- (3) 8,910,451 ratings left. And we split this into a training and a test set by a 7:3 ratio.
- (4) To ease the use of the recsys_metric²[10] library for evaluation, we need to have the same pivoted shape of users and items. Therefore, we chose users who intersect between the train and test sets.

¹<https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data>

²https://github.com/zuoxingdong/recsys_metrics

- (5) The final shape of the pivoted matrix is 13985 users and 5331 items.

2.2 Last.fm 1K

Last.fm 1k³ contains 19,000,000 data points of user-item interactions with 1,000 users and 790,000 songs and has the full listening history of users until May 5th, 2009.[1] Based on the nature of this dataset, which records the number of times a user listens to a song rather than the rating, we made another preparation for the dPF. The following preparation steps are only for PF and BPR models:

- (1) Skip nine rows that are scattered within the data, as those rows contain unique and invalid characters that pandas could not parse.
- (2) Dropped rows that did not have an artist_id, track_id, or timestamp, and any rows that contained Nan.
- (3) Drop unnecessary columns such as track_name, artist_id, and artist_name. Keeping only the timestamp, user_id, and track_id. Leaving us with a dataset that is 16,900,000 rows with 3 columns for user_id, timestamp, and track_id.
- (4) Replace the time stamp values with an int value of 1.
- (5) Convert each user_id and track_id into an int index, which is mapped into the dataset.
- (6) Summed up the number of times a user has interacted with an item and created a new data frame as user_id,item_id, and rating for a number of interactions.
- (7) The columns are swapped into the right ordering of user, item, and rating.
- (8) The resulting shape of the data is 3,957,807 data points with a pivoted matrix of 991 users and 792,004 songs.

2.3 MovieLens 100K

MovieLens 100K[6] is a dataset collected by the individuals at the Grouplens Research Project. The data was collected through the MovieLens website during the seven months from September 19th, 1987, to April 22nd, 1998. MovieLens 100K contains 100,000 ratings from 1-5, 1000 users, and 1700 movies, where each user has rated at least 20 movies. For this dataset, there was no need to prepare the data for PF and BPR.

Data in PF-Last

PF-Last is a unique set of datasets specifically for the PF model, where PF-Last is the last time splice for each dataset. We followed the same format used to define Netflix Time, that is, 16 equal slices. In preparation, we chose to split the prepared dataset by epoch time equally and picked the last 1/16 as our input for the PF:

- (1) Netflix Time
- (2) Netflix Full
- (3) MovieLens 100K
- (4) Last.fm 1K

3 MODEL PREPARATION

3.1 Baseline

The original experiment uses the algorithm Bayesian Poisson Tensor Factorization (BPTF) on the Netflix Time dataset. However,

³<http://ocelma.net/MusicRecommendationDataset/lastfm-1K.html>

due to the hardware limitation and the sheer size of the three-dimensional tensor, we deemed it impossible. As such, we decided to drop this part of the experiment as the algorithm is used once when comparing dPF, BPTF, and PF on the Netflix Time dataset. For the models BPR and PF we use the same number of latent features of 20 as Charlin et al. (2015)

3.2 Dynamic Poisson Factorization

We use and adapt the original code cited in Charlin et al. (2015)[2]. The code produces theta and beta tsv files as user and item feature matrices, a ranking tsv file for the top 100 items predictions.[7] The code also includes several inline metrics like NDCG and precisions, which are not used in this replication because of the avoidance of bias. To achieve accurate results, we bypass the hard limit of the top 100 items by extracting the global user and time feature matrices and computing all-item predictions with a dot product with these matrices.

3.3 Poisson Factorization

The experiment we are trying to replicate references the Hierarchical Poisson Factorization[5] as PF. However, when trying to execute the original code base for HPF, we could not get the dependencies to function in our environment after several attempts. As such, we found another code base similar to HPF, but that differs in that it uses regularization instead of a Bayesian hierarchical structure.[4] The code is also fitted through a gradient-based method instead of a coordinate ascent[9]. Due to the lack of clarity in Charlin et al. (2015), we used the recommended default parameter values. Add

3.4 Bayesian Personalized Ranking

We use the Implicit library⁴ implementation of BPR for our results. Having the learning rate and regularization at 0.001, and the number of iterations at 100. These were the default values for the algorithm.

4 METRICS

As mentioned above, due to hardware limitations, we kept the precision of the results for the Netflix Full dataset at float16, while for the other datasets, we maintained float64 precision. Following Charlin et al. (2015), we set Recall@K with K=50 for consistency. We included Precision@50 and Mean Average Precision(MAP) to evaluate model performance further. Additionally, since the original work did not specify their Top-N selection, we incorporated the Mean Average Rank(MAR) of the top@1000, and all items as a standardized reference based on our experimentation on getting the MRR metrics as close to the original study. Since Charlin et al.(2015) didn't disclose the K value of NDCG in their study, we derived it from their cited code[7] and have K=10. It is worth mentioning that Charlin et al. calculated NDCG using the following formula, which is highly different from today's standard [2].

$$\frac{1}{N} \sum_{i=1}^N \sum_{j \in y_i^N} \frac{1}{\log rank(i, j) + 1} \quad (1)$$

We used the library recsys_metrics[10] for all metrics tests and implemented MAR using Python.

⁴<https://benfred.github.io/implicit/>

Table 1: Netflix-Time

Model	Recall@50	Prec@50	MAP	MAR	MAR@1000	MRR	MRR@1000	NDCG@10
dPF	0.0413	0.0173	0.0475	1222	259	0.0643	0.0507	0.0314
PF	0.0140	0.0453	0.0423	2142	539	0.1421	0.1421	0.0568
PF-last	0.0109	0.0787	0.0873	2153	493	0.3073	0.3073	0.1479
BPR	0.0887	0.2365	0.1711	2000	497	0.4600	0.4600	0.2479

Table 2: Netflix

Model	Recall@50	Prec@50	MAP	MAR	MAR@1000	MRR	MRR@1000	NDCG@10
dPF	0.2126	0.0622	0.1207	1510	340	0.1696	0.0068	0.1282
PF	0.2927	0.0769	0.0959	2659	475	0.2661	0.2661	0.1308
PF-last	0.2906	0.0766	0.0880	2653	548	0.2439	0.2439	0.1198
BPR	0.3715	0.0864	0.1124	2641	457	0.2686	0.2686	0.1478

Table 3: Last.fm

Model	Recall@50	Prec@50	MAP	MAR	MAR@1000	MRR	MRR@1000	NDCG@10
dPF	0.0018	0.0132	0.0431	174347	252	0.0003	0.0294	0.0150
PF	0.0001	0.0013	0.0013	396934	495	0.0084	0.0082	0.0017
PF-last	0.0001	0.0015	0.0015	223843	501	0.0085	0.0083	0.0012
BPR	0.0075	0.0913	0.0243	394811	509	0.2340	0.2340	0.0998

Table 4: MovieLens

Model	Recall@50	Prec@50	MAP	MAR	MAR@1000	MRR	MRR@1000	NDCG@10
dPF	0.2519	0.1032	0.2202	318	243	0.3351	0.3357	0.1617
PF	0.0626	0.0408	0.0417	510	406	0.0772	0.0772	0.0241
PF-last	0.0722	0.0415	0.0469	487	398	0.1638	0.1638	0.0427
BPR	0.2035	0.1084	0.1022	550	328	0.3540	0.3540	0.1565

5 RESULTS

5.1 Replication Findings

The performance of dPF varies across different metrics and datasets. With strong temporal dynamics datasets like Netflix Time, dPF achieves a better MAR score (1222 vs. PF’s 2142), which aligns with Charlin et al.’s hypothesis that temporal modeling improves ranking. However, its Precision@50 (0.0173) lags behind PF (0.0453), while Recall@50 (0.0413) surpasses PF (0.0140). For Netflix, dPF underperformed PF in both Recall@50 (0.2126 vs. 0.2927) and Precision@50 (0.0622 vs. 0.0769), but maintains a higher MAR score (1510 vs. 2659).

Our findings partially validate the original hypothesis that dPF outperforms PF in the general case. In long-term ranking quality, dPF does outperform PF, suggesting its capacity to capture evolving user preferences[2]. However, this advantage could result from a trade-off in short-term ranking accuracy, indicated by poor performance in precision and recall. In our results, PF shows some resilience in the static model when testing against a temporal dataset.

5.2 Comparison

Our results show that PF-Last performs better than PF on MAR@1000, MRR, MRR@1000, and NDCG@10 on the Netflix-Time data set compared to the findings presented by Charlin et al. Additionally, there is a large margin of performance difference between PF and PF-Last on NDCG and MRR compared to what the study shows. The study also indicates that dPF performs the best across the board on Recall@50, MAR, MRR, and NDCG. However, dPF performs worse on MRR, MRR@1000, and NDCG@10.

For Netflix, there are more differences between our results and Charlin et al.’s results, where dPF performs worse on Recall@50, MRR, MRR@1000, and NDCG@10 than PF and PF-Last. The only result that holds for dPF is that it performs better than PF and PF-last on MAR and MAR@1000. The divergence of results continues between our results for PF and PF-last, where PF performance is better than PF-Last on Recall@50, MRR, and MRR@1000 instead of performing worse.

Several factors may lead to discrepancies between our findings and those of Charlin et al. Factors such as:

- (1) The ability to recreate Netflix Time into the same format.
- (2) Using 30% of the Netflix Prize dataset for Netflix instead of 100%
- (3) Different form of HPF

5.3 Additional Findings

Our experiment shows that BPR performs significantly better in the Precision@50, Recall@50, MRR, MRR-100, and NDCG@50 metrics, except on the MovieLens dataset, where it lagged on NDCG@10 and Recall@50 against dPF. The metrics in which BPR consistently performed worse are MAR, MAR-100, and MAP against dPF, except on the Netflix Time dataset, where BPR performed surprisingly well on MAP.

All models struggle with the Last.fm dataset due to its sparsity. However, BPR still leads in many metrics except MAP and MAR, suggesting its good performance in ranking items. Though most metric readings are low for dPF, its high MAP and MAR score show that it has the potential to work with sparsity and have a better average precision.

Models perform quite well on the MovieLens dataset. From the table 3, dPF leads in recall, MAP, and MAR. Additionally, BPR still performs remarkably well in top item predictions, as suggested by precision and MRR, the gap between dPF and BPR is small. One cause of this result could be that MovieLens is a high-density dataset that plays a role in the performance boost.

6 CONCLUSION

We expanded on the experiment by Charlin et al. (2015) by testing dPF against the MovieLens and Last.fm datasets using MAP and precision. In general, our findings yield similarities in MAR results and differences in other metrics.

Overall, we could not fully replicate the experiment conducted by Charlin et al. (2015). Several gaps in information and clarity hindered our ability to achieve total replication, with many critical steps and information inferred from field experiments and third-party sources. This ambiguity includes conflicts in the description of the preparation procedure for the Netflix Time dataset, the missing @k value for NDCG and MRR, and a lack of hints regarding the parameters used for PF.

There are several opportunities to expand this work in the future. We want to explore the performance of BPTF on a smaller dataset with our limited resources and complete the experiment with robust temporal setup metrics. Additionally, we would like to investigate the performance of these models in various domains, particularly those with higher periodic user-item interactions. The majority of our dataset consists of only a single user-item interaction, where a user interacts with an item just once rather than multiple times. We would also like to consider tuning PF and BPR more thoroughly. Overall, the study validates Charlin et al.'s core hypothesis that temporal modeling improves ranking consistency with implicit datasets while revealing some limitations in reproducibility and practicality. Our findings will not significantly undermine previous work, but gaps remain to be filled in future studies.

REFERENCES

- [1] O. Celma. 2010. *Music Recommendation and Discovery in the Long Tail*. Springer.
- [2] Laurent Charlin, Rajesh Ranganath, James McInerney, and David M. Blei. 2015. Dynamic Poisson Factorization. In *Proceedings of the 9th ACM Conference on Recommender Systems (RecSys '15)*. ACM, 155–162. <https://doi.org/10.1145/2792838.2800174>
- [3] David Cortes. 2018. Fast Non-Bayesian Poisson Factorization for Implicit-Feedback Recommendations. *CoRR* abs/1811.01908 (2018). arXiv:1811.01908 <http://arxiv.org/abs/1811.01908>
- [4] David Cortes. 2018. poisfm. <https://github.com/david-cortes/poisfm>.
- [5] Prem Gopalan, Jake M. Hofman, and David M. Blei. 2015. Scalable Recommendation with Hierarchical Poisson Factorization. In *Proceedings of the 31st Conference on Uncertainty in Artificial Intelligence (UAI)*. <https://doi.org/10.1145/1935826.1935845>
- [6] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5, 4 (December 2015), 19. <https://doi.org/10.1145/2827872>
- [7] Icharlin. 2015. DynamicPoissonFactorization. <https://github.com/blei-lab/DynamicPoissonFactorization?tab=readme-ov-file#readme>.
- [8] Bin Li, Xingquan Zhu, Ruijiang Li, Chengqi Zhang, Xiangyang Xue, and Xindong Wu. 2011. Cross-domain collaborative filtering over time. In *Proceedings of the Twenty-Second international joint conference on artificial intelligence—Volume Three*. 2293–2298.
- [9] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. BPR: Bayesian Personalized Ranking from Implicit Feedback. *CoRR* abs/1205.2618 (2012). arXiv:1205.2618 <http://arxiv.org/abs/1205.2618>
- [10] Xingdong Zuo. 2021. *recsys_metrics: An efficient PyTorch implementation of the evaluation metrics in recommender systems*.

A DATA AND CODE AVAILABILITY

- **Evaluations and Data Processing Code:** GitHub (Includes scripts for pre-processing and analysis.)
- **Raw Experiment Results:** Google Drive Folder (Contains datasets and outputs for dPF)